

Curso desarrollo de aplicaciones con tecnologías web
Módulo formativo 2: Programación web en el entorno servidor

Unidad formativa 3: Desarrollo de aplicaciones web distribuidas

Tema 1: Arquitecturas distribuidas orientadas a servicios

Modelo conceptual de las arquitecturas orientadas a servicios (SOA)

Aspectos de seguridad en arquitecturas orientadas a servicios

Modelo conceptual de las arquitecturas orientadas a servicios (SOA)

Las arquitecturas orientadas a servicios (SOA – Service Oriented Architecture) se basan en la idea de dividir una aplicación en **servicios independientes** que se comunican entre sí a través de una red (habitualmente Internet o una intranet).

Objetivo:

Permitir que diferentes sistemas (incluso con tecnologías distintas) colaboren entre sí, facilitando la **reutilización**, la **interoperabilidad** y la **escalabilidad**.

Subapartados del modelo conceptual:

1 Servicios basados en mensajes

- En este modelo, los servicios se comunican enviando **mensajes estructurados** (normalmente XML o JSON).
- Este enfoque es **asíncrono** y adecuado para entornos distribuidos donde las respuestas pueden tardar.

Ejemplo:

Una tienda online lanza un mensaje a un servicio externo de stock preguntando si hay disponibilidad de un producto. El mensaje puede ser procesado de forma asíncrona y la respuesta puede llegar más tarde.

json

```
// Ejemplo de mensaje JSON enviado al servicio
{
  "producto_id": 2025,
  "cantidad": 1
}
```

2 Servicios basados en recursos (REST)

- Se utiliza el protocolo HTTP para acceder a recursos representados como URLs.
- Los servicios RESTful usan métodos HTTP estándar:
- GET (leer datos)
- POST (crear)
- PUT (actualizar)
- DELETE (eliminar)

Ejemplo real:

Un sistema de reservas accede a:

GET <https://api.hoteles.com/reservas/12345>

Y recibe:

```
{  
  "id": 12345,  
  "cliente": "Juan Pérez",  
  "fecha": "2025-04-10",  
  "estado": "confirmada"  
}
```

3 Políticas y contratos de servicios

- Las **políticas** definen reglas de uso de un servicio: seguridad, tiempos de espera, autenticación, etc.
- Los **contratos** definen qué datos espera un servicio y qué devuelve.

Normalmente expresado en un lenguaje como **WSDL** para servicios SOAP, o en la documentación para APIs REST.

Ejemplo:

Un contrato puede definir que al enviar un POST /usuarios se espera un JSON con nombre y correo, y se devuelve un objeto usuario con ID y fecha de creación.

En resumen:

Modelo	Comunicación	Formato	Ejemplo
Basado en mensajes	Envío de datos entre servicios (asíncrono)	XML / JSON	Cola de mensajes, Kafka
Basado en recursos	Peticiones HTTP a URLs (REST)	JSON	API REST de Stripe, Twilio
Contratos y políticas	Documentan cómo usar los servicios	WSDL / OpenAPI / RAML	API Docs de GitHub

Ejercicio práctico:

Elige una API pública (como la de **OpenWeather** o **TheCatAPI**) y describe:

1. Si está basada en mensajes o recursos.
2. Qué contrato tiene (parámetros y respuesta).
3. Si hay políticas de uso (autenticación, límites, etc.).

Aspectos de seguridad en arquitecturas orientadas a servicios

En los entornos distribuidos, la **seguridad** es un aspecto crítico. Los servicios pueden estar expuestos a la red pública, y es necesario proteger tanto los **datos**, como el **acceso** y las **comunicaciones**.

1 Seguridad de datos

Se refiere a proteger los datos almacenados y transmitidos. Los riesgos incluyen:

- Pérdida o robo de datos
- Modificación no autorizada
- Acceso indebido

Soluciones:

- Cifrado de datos en tránsito (HTTPS) y en reposo (base de datos cifrada)
- Uso de tokens o claves para identificar al usuario

2 Seguridad de mensajes

Especialmente relevante en servicios **basados en mensajes** (como SOAP o colas de mensajes).

Requiere garantizar:

- **Integridad:** que el mensaje no fue alterado
- **Autenticación:** quién envió el mensaje
- **No repudio:** que el emisor no pueda negar haberlo enviado

Tecnologías:

- XML Encryption, XML Signature (en SOAP)
- JWT (JSON Web Tokens) en servicios REST
- HTTPS + cabeceras de autenticación

3 Control de acceso. El modelo RBAC

RBAC (Role-Based Access Control) define el acceso a recursos en función del **rol** del usuario.

Ejemplo de roles:

- Admin → puede leer, escribir, borrar
- Usuario → solo puede leer y escribir
- Invitado → solo puede leer

Ejemplo en código PHP:

```
if ($usuario->rol === 'admin') {  
    // mostrar opciones avanzadas  
} else {  
    // acceso restringido  
}
```

4 Seguridad en comunicaciones. Protocolos seguros

Las comunicaciones entre servicios deben estar protegidas para evitar **intercepciones**, **man-in-the-middle** y otros ataques.

Protocolos comunes:

- **HTTPS (HTTP + SSL/TLS):** garantiza cifrado de extremo a extremo
- **VPNs:** para redes internas seguras
- **OAuth2:** para autorización segura entre aplicaciones (muy usado en REST APIs)

Ejemplo real:

- Una aplicación de gestión usa una API externa de pagos (Stripe). La API exige:
 - HTTPS
 - Autenticación por token secreto
 - Límite de peticiones por minuto (política de uso)

En resumen:

Seguridad	Objetivo	Solución típica
Datos	Proteger lo almacenado y transmitido	Cifrado, HTTPS
Mensajes	Asegurar integridad y autenticidad	Firmas, tokens
Acceso	Controlar quién puede hacer qué	RBAC, OAuth2
Comunicación	Evitar ataques y filtraciones	SSL/TLS, VPN

Ejercicio práctico:

Imagina una aplicación que permite a profesores subir notas de sus alumnos.

1. ¿Qué roles implementarías?
2. ¿Qué medidas de seguridad aplicarías para proteger los datos?
3. ¿Cómo asegurarías que un mensaje enviado desde un panel remoto no fue modificado?